Dynamic Programming

Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.

Dynamic programming is applicable when the subproblems share subproblems.

In this context, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the common subproblems.

A dynamic-programming algorithm solves every subproblem just once and then saves its answer in a table, thereby avoiding the work of re-computing the answer every time the

subproblem is encountered.

Basic elements of DP

Substructure: Decompose your problem into smaller (and hopefully simpler) subproblem. Express the solution of the original problem in terms of solutions for smaller problems.

Table-structure: Store the answers to the sub-problems in a table. This is done because subproblem solutions are reused many times.

Bottom-up computation: Combine solutions on smaller subproblems to solve larger subproblems.

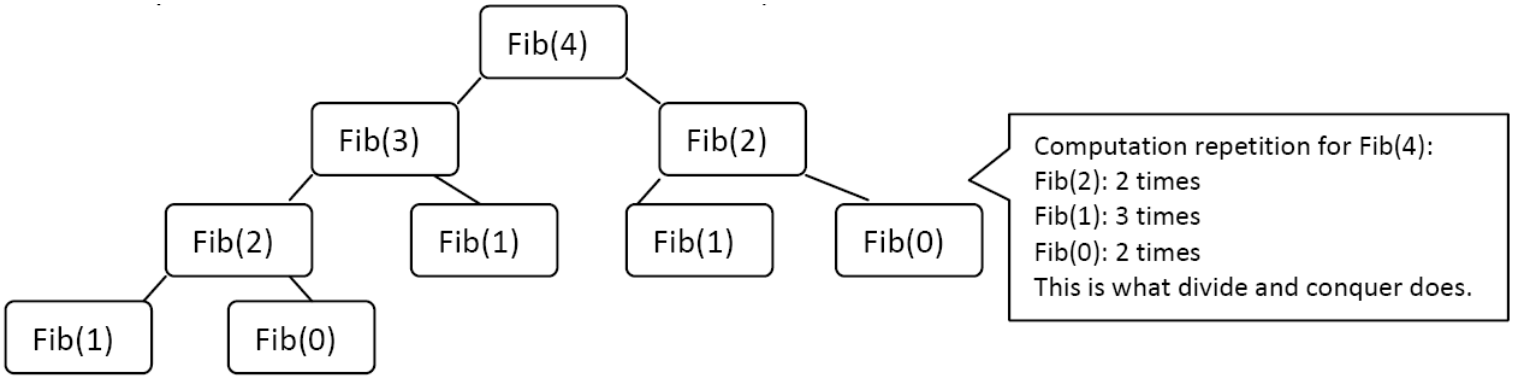
(There is also top-down alternative, called memoization)

DP is not applicable to all optimization problems. There are two important elements that a problem must have in order for DP to be applicable.

Optimal substructure (Principle of optimality): It states that for the problem to be solved global optimally, each sub problem should be solved optimally.

Polynomial many sub problems: An important aspect to the efficiency of DP is that the total number of subproblems to be solved should be at most a polynomial number. (Like n2, n3…)

Fibonacci Numbers



0/1 Knapsack Problem

Statement: A thief has a bag or knapsack that can contain maximum weight W of his loot. There are n items and the weight of i^{th} item is w_i and it worth v_i. We can either pick item or leave it i.e. x_i is 0 or 1. Here the objective is to collect the items that maximize the total profit earned.
We can formally state this problem as, maximize $\sum_{i=1}^{n} x_i v_i$ using constraints $\sum_{i=1}^{n} x_i w_i \leq W$.

The algorithm takes as input maximum weight W, the number of items n, two arrays v[] for values of items and w[] for weight of items and returns a table c as output.
Let us assume that the table-cell c[i,w] is the value of solution for items 1 to i and maximum weight w (small w). Then we can define recurrence relation for 0/1 knapsack problem as:

$$c[i,w] = \begin{cases} 0 & if\ i = 0\ or\ w = 0 \\ c[i-1,w] & if\ w_i > w \\ Max\{v_i + C[i-1,w-w_i], c[i-1,w] & if\ i > 0\ and\ w \geq w_i \end{cases}$$

Algorithm:

```
DP_01Knapsack(W, n , v[], w[])
{
    Create new 2-d array (table) c[n,w]
    for(w = 0; w <= W; w++)
        c[0,w] = 0;
    for(i = 1; i <= n; i++)
        c[i,0] = 0;
    for(i = 1; i <= n; i++)
    {
        for(w = 1; w <= W; w++)
        {
                if(w[i] <= w)
                {
                        if (v[i] +c[i-1,w-w[i]] > c[i-1,w])
                                c[i,w] = v[i] +c[i-1,w-w[i]];
                        else
                                c[i,w] = c[i-1,w];
                }
                else
                        c[i,w] = c[i-1,w];
        }
    }
}
```

$$c[i,w] = \begin{cases} 0 & if\ i = 0\ or\ w = 0 \\ c[i-1,w] & if\ w_i > w \\ Max\{v_i + C[i-1,w-w_i], c[i-1,w]\} & if\ i > 0\ and\ w \geq w_i \end{cases}$$

T(n)= O(n*W)

S(n)= O(n*W)

Example n=7 items where v[] = {2,3,3,4,4,5,7}and w[] = {3,5,7,4,3,9,2}and W = 9.

$$c[i,w] = \begin{cases} 0 & if\ i = 0\ or\ w = 0 \\ c[i-1,w] & if\ w_i > w \\ Max\{v_i + C[i-1,w-w_i], c[i-1,w] & if\ i > 0\ and\ w \geq w_i \end{cases}$$

i/w	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	2	2	2	2	2	2	2
2	0	0	0	2	2	3	3	3	5	5
3	0	0	0	2	2	3	3	3	5	5
4	0	0	0	2	4	4	4	6	6	7
5	0	0	0	4	4	4	6	8	8	8
6	0	0	0	4	4	4	6	8	8	8
7	0	0	7	7	7	11	11	11	13	15

Example:
Let, number of items n = 6, there weights be
w[] = {2, 3, 6, 5, 4, 1} and corresponding values be
v[] = {13, 14, 24, 25, 18, 10}.
If knapsack can hold maximum weight W = 8, we have to find items that gives us maximum benefit.

Given: w[] = {2, 3, 6, 5, 4, 1}, v[] = {13, 14, 24, 25, 18, 10}

i\w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	13	13	13	13	13	13	13
2	0	0	13	14	14	27	27	27	27
3	0	0	13	14	14	27	27	27	37
4	0	0	13	14	14	27	27	38	39
5	0	0	13	14	18	27	31	38	39
6	0	10	13	23	24	28	37	41	48

To find items to keep,
Start with i = n and j = W
If c[i, j] != c[i-1, j]
Take ith item
i = i – 1
j = j – w[i]
else i = i – 1

$$c[i,w] = \begin{cases} 0 & if\ i = 0\ or\ w = 0 \\ c[i-1,w] & if\ w_i > w \\ Max\{v_i + C[i-1,w-w_i], c[i-1,w] & if\ i > 0\ and\ w \geq w_i \end{cases}$$

Matrix-Chain-Multiplication

Dynamic programming is an algorithm that solves the problem of matrix-chain multiplication. We are given a sequence (chain) (A1; A2;…;An) of n matrices to be multiplied, and we wish to compute the product A1.A2...An.

If the chain of matrices is (A1; A2; A3; A4), then we can fully parenthesize the product A1A2A3A4 in five distinct ways:

- (A1.(A2.(A3A4))) ;
- (A1((A2A3)A4)) ;
- ((A1A2).(A3.A4));
- (((A1.A2)A3).A4) ;
- ((A1(A2A3))A4) .

These sequence have different scalar multiplication cost.

Dynamic programming chooses minimum among them.

Recursive definition

Let m[i, j] denotes minimum number of scalar multiplications needed to compute $A_{i \ldots j}$.

$$m[i,j] = \begin{cases} 0 & \text{if } i = j \ , \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1} p_k p_j\} & \text{if } i < j \ . \end{cases}$$

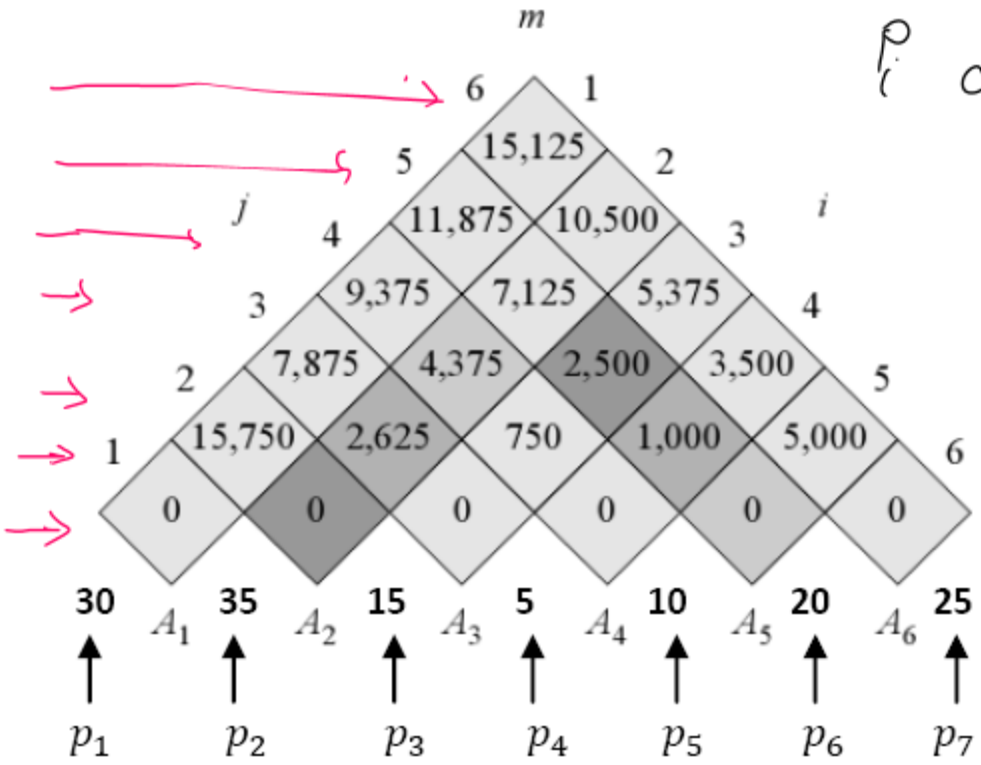
MATRIX-CHAIN-ORDER(p)

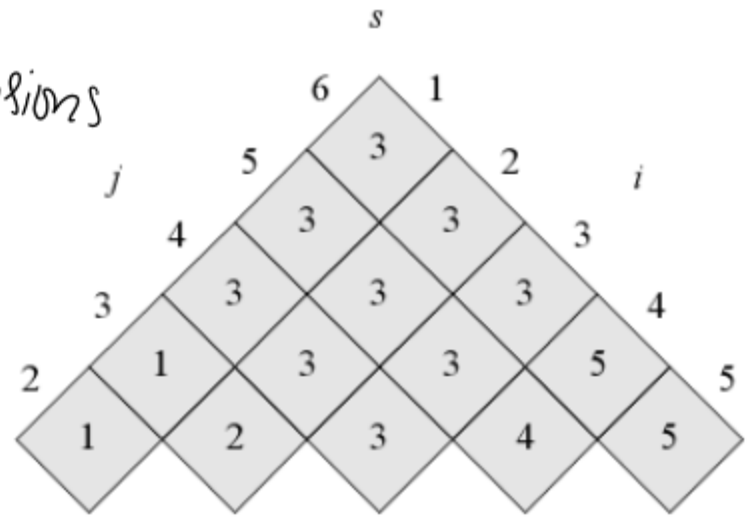
```
1   n = p.length − 1
2   let m[1..n, 1..n] and s[1..n − 1, 2..n] be new tables
3   for i = 1 to n
4       m[i, i] = 0
5   for l = 2 to n                  // l is the chain length
6       for i = 1 to n − l + 1
7           j = i + l − 1
8           m[i, j] = ∞
9           for k = i to j − 1
10              q = m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
11              if q < m[i, j]
12                  m[i, j] = q
13                  s[i, j] = k
14  return m and s      // minimum cost table  and parenthesis order info table.
```

Example: let matrices and their dimensions be A_1 A_2 A_3 A_4 A_5 A_6

30X35 35X15 15X5 5X10 10X20 20X25

0 1 1 2 2 3 3 4 4 5 5 6





$$m[i,j] = \begin{cases} \min_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1} p_k p_j\} \end{cases}$$

Tables are rotated so that a diagonal lies horizontally. Each table cells are calculated as:

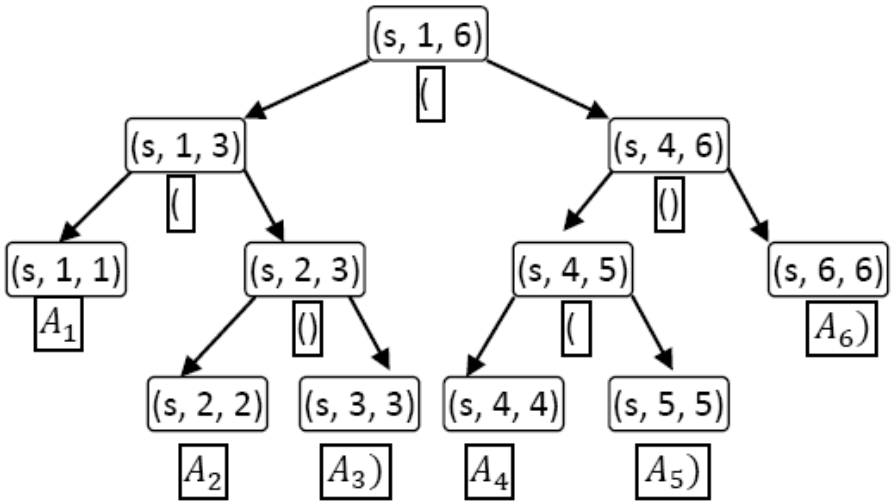
$$m[1,2] = \sum_{1<k\le 2} \min\{m[1,k] + m[k+1,2] + p_{1-1} p_k p_2\}$$
$$= \min\{m[1,1] + m[2,2] + p_0 p_1 p_2\}$$
$$= \min\{0 + 0 + 30 \times 35 \times 15\}$$
$$= 15750$$ (also s[1, 2] = 1 (k for which we get minimum value))

$k = 1$

$$m[l, j] = \begin{cases} \min_{i \le k < j} \{m[i,k] + m[k+1, j] + pi\_{1} p_k p_j\} \end{cases}$$

$$m[2,5] = min \begin{cases} m[2,2] + m[3,5] + p1p2p5 = 0 + 2500 + 35.15.20 = 13000, \\ m[2,3] + m[4,5] + p1p3p5 = 2625 + 1000 + 35.5.20 = 7125, \\ m[2,4] + m[5,5] + p1p4p5 = 4375 + 0 + 35.10.20 = 11375 \end{cases}$$

Now, we use *s* table to fully parenthesize matrix chain:



Recursion tree is for **PRINT-OPTIMAL-PARENS (s, i, j)** where nodes has only argument set and two nodes for two recursive calls.

Tracing Rule:

- Each left node (including root) where $i \neq j$prints "(" and prints a Matrix A_i if $i = j$.
- Each right node where $i \neq j$prints "()" and if $i = j$, prints "A_i)".

OUTPUT: $((A_1(A_2A_3))((A_4A_5)A_6))$

Analysis

Time Complexity: *Three nested for loops*

Space complexity : *Size of matrix*

String Editing Algorithm

String Editing Algorithm(edit distance problem with insertion, deletion, replace operation) and its complexity analysis

Recursive definition

Let Cost(i,j) be the cost of transforming X=x1,x2,……..xi into Y=y1,y2……yj, D(xi) be the cost of deleting xi from X, I(xi) be the cost of inserting symbol xi into X and C(xi,yj) be the cost of changing symbol xi into yj.

$$\text{Cost}(i,j) = \begin{cases} 0 & \text{if } i = j = 0 \\ \text{Cost}(i-1,0) + D(xi) & \text{if } j=0 \text{ and } i>0 \\ \text{Cost}(0,j-1) + I(yj) & \text{if } i=0 \text{ and } j>0 \\ \text{Cost'}(i,j) & \text{if } i>0, j>0 \end{cases}$$

$$\text{Where Cost'}(i,j) = \min \{ \text{Cost}(i-1,j) + D(xi),\ \text{Cost}(i,j-1) + I(yj),\ \text{Cost}(i-1,j-1) + C(xi,yj) \} \quad \text{if } i \neq 0 \text{ and } j \neq 0$$

$C(xi,yj) = 0$ if $xi = yj$ and $C(xi,yj) = 2$ if $xi \neq yj$

Recursive definition

$$\text{Cost}(i, j) = \begin{cases} 0 & \text{if } i = j = 0 \\ \text{Cost}(i-1,0) + D(xi) & \text{if } j=0 \text{ and } i>0 \\ \text{Cost}(0,j-1) + I(yj) & \text{if } i=0 \text{ and } j>0 \\ \text{Cost'}(i,j) & \text{if } i>0,j>0 \end{cases}$$

Where $\text{Cost'}(i, j) = \min \{ \text{Cost}(i-1,j) + D(xi),\ \text{Cost}(i,j-1) + I(yj),\ \text{Cost}(i-1,j-1) + C(xi,yj) \}$ if $i \neq 0$ and $j \neq 0$

$C(xi,yj) = 0$ if $xi = yj$ and $C(xi,yj) = 2$ if $xi \neq yj$

Transform the String X=aabab into Y=babb

I \ J	0	1 b	2 a	3 b	4 b
0	0	1	2	3	4
1 a	1	2	1	2	3
2 a	2	3	2	3	4
3 b	3	2	3	2	3
4 a	4	3	2	3	4
5 b	5	4	3	2	3

Edit Distance=C[5][4]=3

String Editing Algorithm

Algorithm:

```
StringEditing(x,y)
{
        n=length(x);    m=length(y);
        for(j=0;j<=m;j++)
           C[0][j]=j;
        for(i=0;i<=n;i++)
           C[i][0]=i;
        for(i=1;i<=n;i++)
            for(j=1;j<=m;j++)
             {
                        c1=C[i-1][j]+1; c2=C[i][j-1]+1;
                        if(xi== yj)
                                c3=C[i-1][j-1];
                        else
                                c3=C[i-1][j-1]+2;
                        C[i][j]= min{c1,c2,c3};
              }
}
```

Analysis:

Time complexity = $O(nm) = O(n^2)$

Floyd Warshall Algorithms

- Floyd Warshall Algorithms for all pair shortest path problem, example and its complexity analysis.

Floyd Warshall Algorithms

Floyd-Warshall is all pairs shortest path algorithm can be solved using dynamic-programming on a directed graph G=(V,E). There may be negative weighted edges but no negative weighted cycle.

For path p=(v1,v2……vl), an intermediate vertex is any vertex of p other than v1 or vl.

Let $d_{ij}^{(k)}$ = shortest path weight of any path i to j with all intermediate vertices in {1,2,..k}

If k is not an intermediate vertex in i to j, then we do not update $d_{ij}^{(k)}$.

If k is an intermediate vertex of shortest path form i to j, update $d_{ij}^{(k)}$.

Recursive formula:

When k=0, a path from vertex i to vertex j has no intermediate vertices at all. Such a path has at most one edge, and hence $d_{ij}^{(0)} = w_{ij}$.

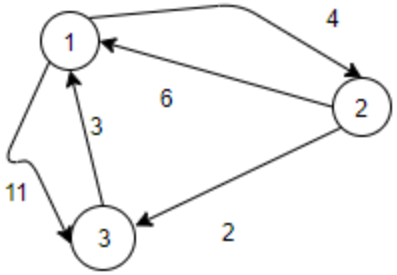
Because for any path, all intermediate vertices are in the set{1,2…n}, the matrix $D^{(n)}=d_{ij}^{(n)}$ gives the final answer: $d_{ij}^{(n)}= \delta(i,j)$ for all $i, j \in V$

Algorithm

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0\,, \\ \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right) & \text{if } k \geq 1\,. \end{cases}$$

```
FLOYD-WARSHALL(W)
1  n = W.rows
2  D^(0) = W
3  for k = 1 to n
4       let D^(k) = (d_ij^(k)) be a new n × n matrix
5       for i = 1 to n
6            for j = 1 to n
7                 d_ij^(k) = min(d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1))
8  return D^(n)
```

Floyd Warshall Algorithms



$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0 , \\ \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right) & \text{if } k \geq 1 . \end{cases}$$

D0	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

D1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

D2	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

D3	1	2	3
1	0	4	6
2	5	0	2
3	3	4	0

$D^1(2,3)=\min\{ D^0(2,3), D^0(2,1)+D^0(1,3)\}$

$=\min\{2,6+11\}$

$=2$

$D^1(3,2)=\min\{ D^0(3,2), D^0(3,1)+D^0(1,2)\}$

$=\min\{\infty,3+4\}$

$=7$

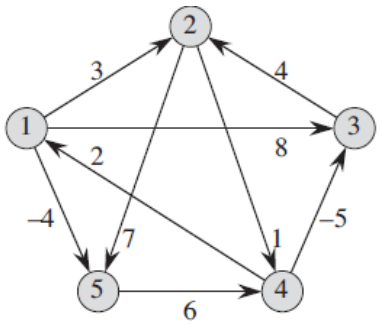
$D^2(1,3)=$

$D^2(3,1)=$

$D^3(1,2)=$

$D^3(2,1)=$

Floyd Warshall Algorithms



$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

Travelling Salesman Problem

Travelling Salesman Problem and its analysis

Travelling Salesman Problem

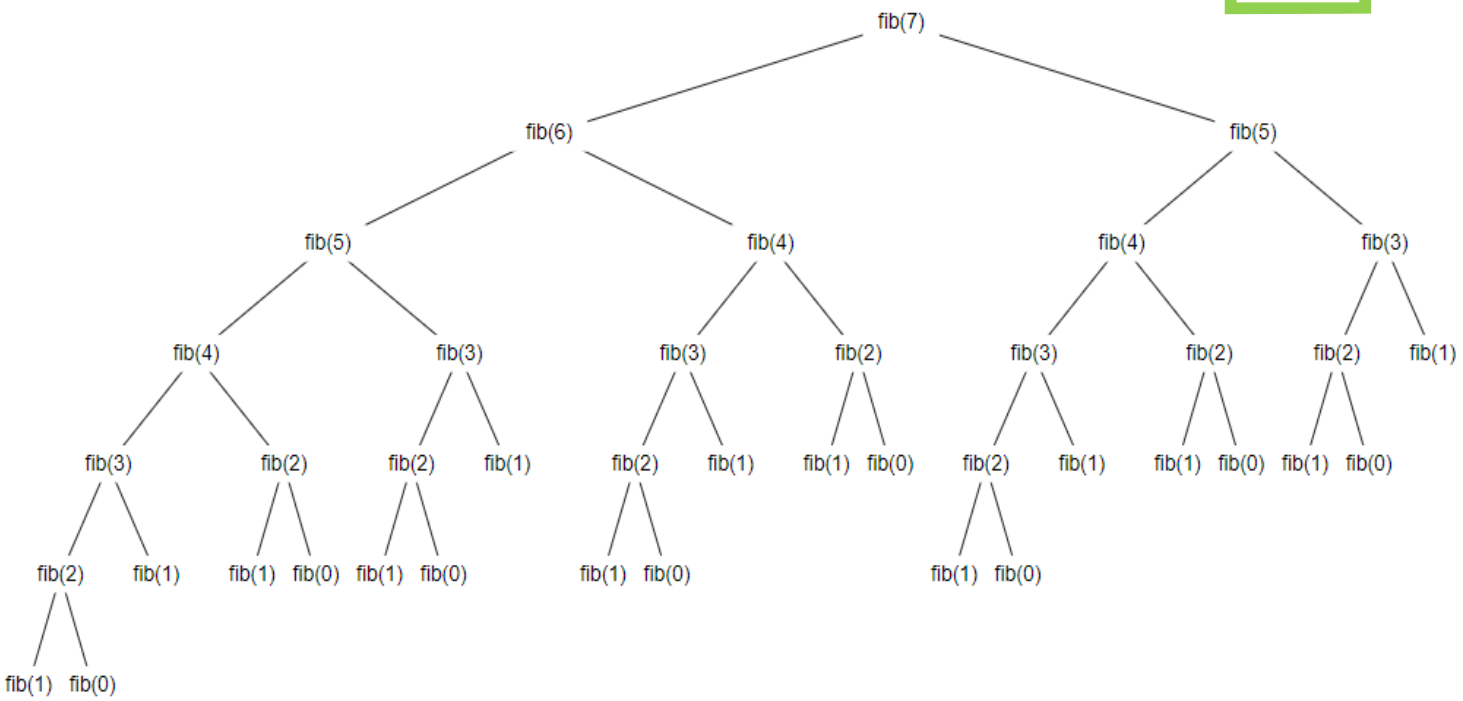
Memoization Strategy

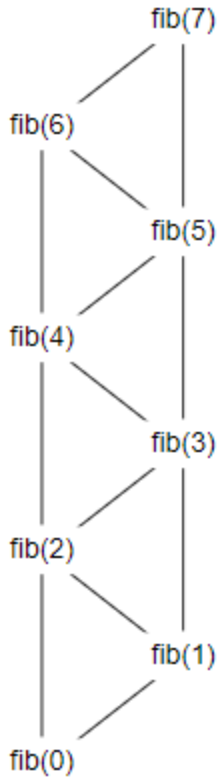
Dynamic programming is a technique for solving problems recursively.
It can be implemented by **memoization** or **tabulation.**

If the original problem requires all subproblems to be solved, tabulation usually outperformes memoization by a constant factor. This is because tabulation has no overhead for recursion and can use a preallocated array rather than, say, a hash map.
If only some of the subproblems need to be solved for the original problem to be solved, then memoization is preferrable since the subproblems are solved lazily, i.e. precisely the computations that are needed are carried out.

Memoization Strategy

Dynamic programming is a technique for solving problems recursively.
It can be implemented by **memoization** or **tabulation**.
Memoization refers to the technique of caching and reusing previously computed results.





Calculation of f(n)

```
fib_mem(n)
{
   if (mem[n] is not set)
      if (n < 2) result = n
      else result = fib_mem(n-2) + fib_mem(n-1)
      mem[n] = result
   return mem[n]
}
```

```
fib_mem(4)
fib_mem(3)
fib_mem(2)
fib_mem(1)
fib_mem(0)
fib_mem(1) already computed
fib_mem(2) already computed
```